Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Sayed Afras S
Register Number	3122247001059
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	14.08.2026

1 Objective

The aim of this experiment is to see how different machine learning models behave when they are trained on the same classification data, and what changes when the number of input features is reduced using PCA. The notebook compares models such as SVM, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and Stacking.

A second part of the experiment is the PCA comparison. The original dataset has 30 numerical features. PCA is used to reduce this to 10 components while keeping at least 95% of the total variance. The models are then evaluated with and without PCA using 5-fold stratified cross-validation. This makes the comparison more useful than checking only one train-test split.

2 Problem Statement

The problem is a binary classification task on breast cancer data. Each sample has numerical measurements from breast mass images, and the model has to identify the sample as malignant or benign.

Instead of stopping with one classifier, the notebook tries ten different models. Their main hyperparameters are searched using `GridSearchCV`. The score used for model selection is F1-score, since it gives a better balance between the two parts of the classification result than accuracy alone.

The experiment also asks a simple practical question: **does PCA actually help?** A smaller feature space should make the model simpler, but reducing dimensions can also remove information that some models use. The notebook checks both cases before drawing a conclusion.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	Scikit-learn built-in breast cancer dataset (originally from UCI)
Number of Samples	569
Number of Features	30 numerical features
Number of Classes	2 – Malignant and Benign
Target Encoding	0 = malignant, 1 = benign
Class Counts	212 malignant, 357 benign

The dataset has 569 samples. There are 357 benign samples and 212 malignant samples, so the classes are not perfectly balanced. The imbalance is not very large, but it is still worth keeping in mind while reading the validation scores.

4 Exploratory Data Analysis

The notebook first checks the shape of the feature matrix and the target distribution. The output is `(569, 30)`, which confirms that there are 30 input features for every sample.

The target count shows 357 samples in class 1 and 212 samples in class 0. So, the benign class is larger. There is no separate missing-value treatment in the code because the built-in dataset is already numeric and does not require missing-value handling here.

Another practical point is the scale of the features. Some measurements are much larger than others. For example, area-related features can be in the hundreds, while smoothness-type features are much smaller. Scaling is therefore important, especially for models such as SVM and KNN.

5 Data Preprocessing

The notebook uses `StandardScaler` so that the input features have approximately zero mean and unit variance. After scaling, PCA is fitted and the smallest number of components that reaches 95% cumulative explained variance is selected.

Listing 1: Feature scaling and PCA part used in the notebook

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca_full = PCA(random_state=RANDOM_STATE).fit(X_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
n_components_95 = np.argmax(cum_var >= 0.95) + 1

pca_95 = PCA(n_components=n_components_95,
             random_state=RANDOM_STATE)
X_pca = pca_95.fit_transform(X_scaled)
```

The selected number of components is 10, and these components explain 95.16% of the variance. So the feature count goes from 30 to 10 without throwing away most of the variation in the data.

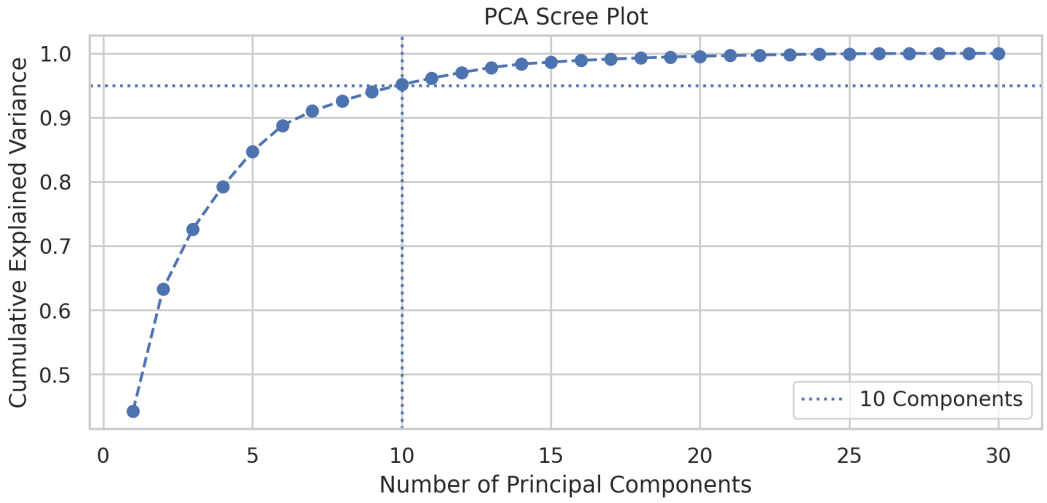

Figure 1: PCA cumulative explained variance and the 95% threshold

Inference: The curve crosses the 95% line at the 10th component. After that point the increase becomes much smaller. So using 10 components is a reasonable reduction for this dataset.

6 Machine Learning Algorithms

The notebook configures ten classifiers. The idea is to include both simple models and ensemble-style models so that their behaviour can be compared on the same data.

6.1 Support Vector Machine

SVM tries to find a decision boundary that separates the two classes with a good margin. The notebook tests linear and RBF kernels along with different values of C and γ.

6.2 Naive Bayes

Gaussian Naive Bayes is a simple probabilistic classifier. It is fast and works directly with the numerical features, but its probability assumptions may not fit every dataset perfectly.

6.3 K-Nearest Neighbours

KNN predicts a sample by looking at nearby training points. The notebook changes the number of neighbours, distance metric and whether uniform or distance-based weights are used.

6.4 Logistic Regression

Logistic Regression gives a linear decision boundary and predicts the probability of each class. The notebook searches the regularization parameter C and two solvers.

6.5 Decision Tree

A Decision Tree creates rule-like splits on the features. It is easy to understand, but a single tree can become sensitive to the training data. The search therefore changes the tree depth and split criterion.

6.6 Random Forest

Random Forest combines many decision trees. The main parameters explored here are the number of trees and maximum tree depth. This is the bagging-style model in the notebook.

6.7 AdaBoost and Gradient Boosting

AdaBoost and Gradient Boosting build models in sequence rather than making all models independent. Each new stage tries to improve the current errors. Their main search

parameters are the number of estimators, learning rate and, for Gradient Boosting, tree depth.

6.8 XGBoost

XGBoost is another boosting approach used in the notebook. The search changes the number of estimators, learning rate and maximum depth.

6.9 Stacking

The Stacking model uses KNN, Gaussian Naive Bayes and a Decision Tree as base learners. Logistic Regression is used as the final estimator. This is useful because the base models do not learn the data in exactly the same way, so their outputs can complement each other.

7 Hyperparameter Selection

A stratified 5-fold split is used so that the class ratio is kept reasonably similar across folds. For every model, `GridSearchCV` searches the parameter combinations and uses F1-score as the scoring value.

Model	Parameters explored	Values
SVM	C, γ, kernel	$C \in \{0.1, 1, 10\}$; scale/auto; RBF/linear
Naive Bayes	var_smoothing	$10^{-9}, 10^{-8}, 10^{-7}$
KNN	neighbors, weights, metric	3,5,7,9; uniform/distance; Euclidean/Manhattan
Logistic Regression	C, solver	0.01,0.1,1,10; lbfgs/liblinear
Decision Tree	max_depth, criterion	3,5,10,None; gini/entropy
Random Forest	n_estimators, max_depth	50,100; 3,5,10
AdaBoost	n_estimators, learning_rate	30,50,100; 0.01,0.1,1.0
Gradient Boosting	n_estimators, learning_rate, max_depth	50,100; 0.01,0.1; 3,5
XGBoost	n_estimators, learning_rate, max_depth	50,100; 0.01,0.1; 3,5
Stacking	meta learner C	0.1,1.0,10.0

8 5-Fold Cross-Validation Results

The main comparison from the notebook is the F1-score of each fold. Two versions are shown: the original 30-feature input and the 10-component PCA input.

8.1 Without PCA

Table 1: Five-fold F1-score without PCA

Model	F1-1	F1-2	F1-3	F1-4	F1-5	Average
SVM	0.9929	0.9718	0.9730	0.9930	0.9861	0.9834
Naive Bayes	0.9650	0.9315	0.9467	0.9197	0.9583	0.9442
KNN	0.9930	0.9660	0.9660	0.9859	0.9660	0.9754
Logistic Regression	0.9859	0.9726	0.9730	0.9930	0.9930	0.9835
Decision Tree	0.9429	0.9324	0.9793	0.9571	0.9444	0.9512
Random Forest	0.9714	0.9524	0.9660	0.9565	0.9790	0.9651
AdaBoost	0.9930	0.9517	0.9660	0.9787	0.9793	0.9738
Gradient Boosting	0.9577	0.9722	0.9660	0.9650	0.9510	0.9624
XGBoost	0.9857	0.9524	0.9726	0.9787	0.9790	0.9737
Stacking	0.9861	0.9660	0.9595	0.9859	0.9660	0.9727

The best average F1 without PCA is Logistic Regression at 0.9835, followed very closely by SVM at 0.9834. Naive Bayes has the lowest average in this run at 0.9442.

8.2 With PCA

Table 2: Five-fold F1-score with 10 PCA components

Model	F1-1	F1-2	F1-3	F1-4	F1-5	Average
SVM	0.9930	0.9660	0.9730	0.9930	0.9861	0.9822
Naive Bayes	0.9718	0.9231	0.9221	0.9091	0.9379	0.9328
KNN	0.9930	0.9660	0.9595	0.9790	0.9861	0.9767
Logistic Regression	0.9859	0.9793	0.9730	0.9859	0.9930	0.9834
Decision Tree	0.9787	0.9517	0.9388	0.9333	0.9718	0.9549
Random Forest	0.9859	0.9388	0.9660	0.9420	0.9718	0.9609
AdaBoost	0.9714	0.9583	0.9730	0.9640	0.9859	0.9705
Gradient Boosting	0.9857	0.9726	0.9664	0.9565	0.9718	0.9706
XGBoost	1.0000	0.9589	0.9530	0.9640	0.9787	0.9709
Stacking	0.9790	0.9660	0.9589	0.9714	0.9861	0.9723

With PCA, Logistic Regression is again the top model with an average F1 of 0.9834. SVM is next at 0.9822. The PCA version of Naive Bayes drops to 0.9328. This is one of the clearest examples here that dimensionality reduction does not give the same benifit to every classifier.

9 PCA vs. No-PCA Comparison

Table 3: Average 5-fold F1 comparison

Model	No-PCA	With-PCA	Change
SVM	0.9834	0.9822	-0.0012
Naive Bayes	0.9442	0.9328	-0.0114
KNN	0.9754	0.9767	+0.0013
Logistic Regression	0.9835	0.9834	-0.0001
Decision Tree	0.9512	0.9549	+0.0037
Random Forest	0.9651	0.9609	-0.0042
AdaBoost	0.9738	0.9705	-0.0033
Gradient Boosting	0.9624	0.9706	+0.0082
XGBoost	0.9737	0.9709	-0.0028
Stacking	0.9727	0.9723	-0.0004

The biggest positive change is seen in Gradient Boosting, whose average F1 increases from 0.9624 to 0.9706. KNN and Decision Tree also improve a little. On the other hand, Naive Bayes drops the most. For SVM, Logistic Regression and Stacking, the difference is very small. So, PCA is not automatically better or worse; its effect depends on the model.

10 Result Visualization

10.1 Confusion Matrices

The notebook visualizes SVM and Gradient Boosting for both feature sets. The matrices below are generated from the selected models after fitting them to the full available dataset, matching the code in the notebook.

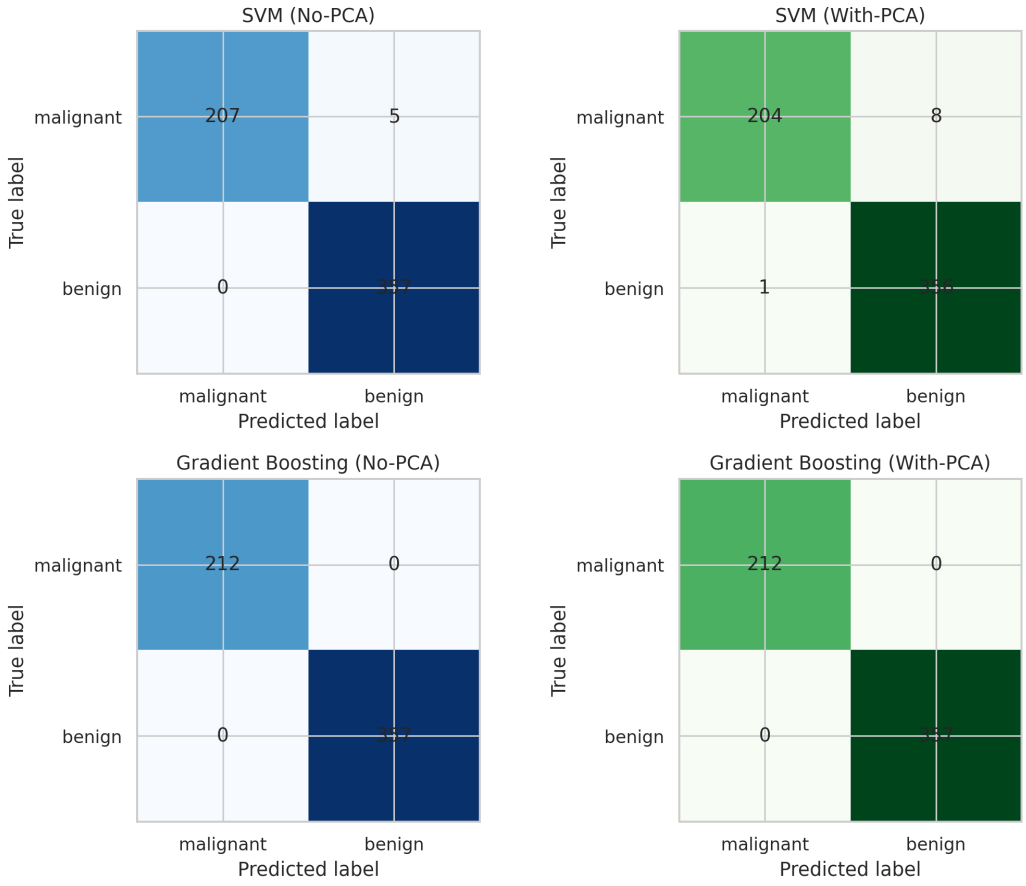

Figure 2: Confusion matrices for SVM and Gradient Boosting with and without PCA

Inference: SVM makes a few more classification mistakes after PCA, while Gradient Boosting stays very strong in both cases. In the notebook run, Gradient Boosting gives an especially clean matrix on these fitted samples. This visual check also shows that the effect of PCA is not the same for all models.

10.2 ROC Curve

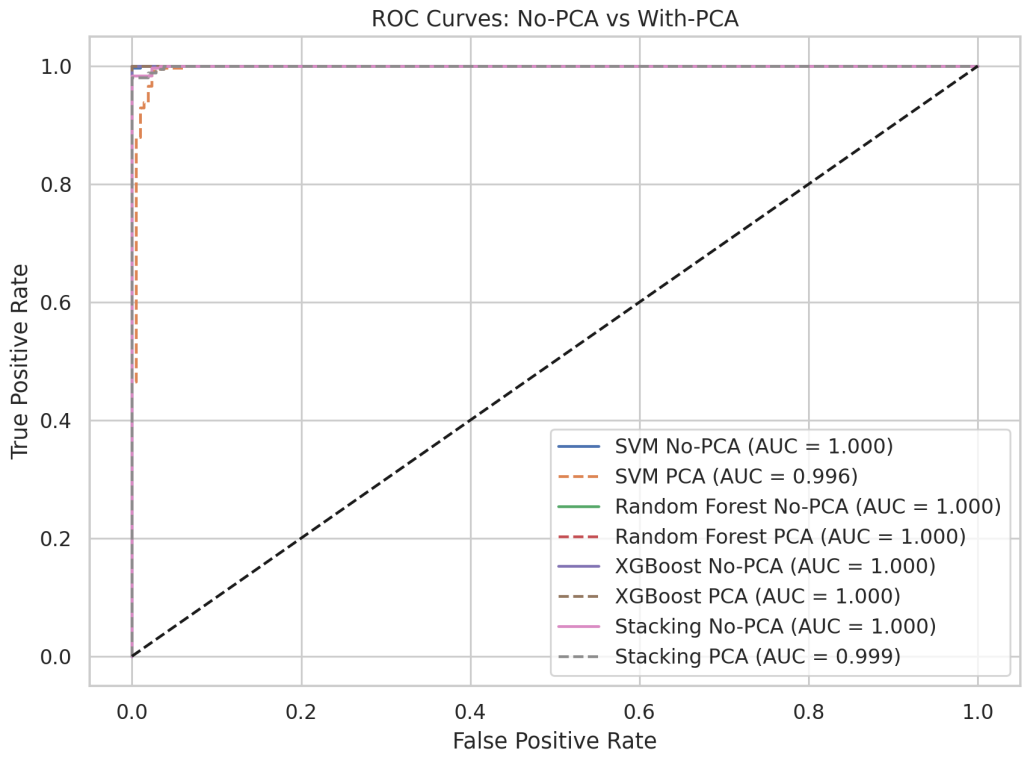

Figure 3: ROC curves comparing selected models before and after PCA

Inference: The ROC curves are close to the top-left corner for all four selected models. The no-PCA SVM, Random Forest, XGBoost and Stacking curves are extremely high on the fitted data, while the PCA versions remain very close. This means that the reduced feature representation still keeps strong class separation for these models.

11 Discussion

The main result is that reducing 30 features to 10 components does not destroy the overall classification performance. In fact, some models improve slightly after PCA. Gradient Boosting gains the most in average F1 among the models shown, going from 0.9624 to 0.9706.

At the same time, the change is not universal. Naive Bayes falls from 0.9442 to 0.9328. Random Forest also drops a little. SVM and Logistic Regression are almost unchanged. This tells me that PCA is more of a model-dependent choice than a fixed preprocessing rule.

Without PCA, Logistic Regression gives the highest average F1 of 0.9835. With PCA it is still almost the same at 0.9834. SVM is also very close. So, for this dataset, PCA helps reduce the feature count a lot, but it does not provide a major F1 improvement for the strongest models.

One limitation in the notebook is worth mentioning. The scaler and PCA are fitted once before the 5-fold cross-validation, using all 569 samples. That is not the cleanest validation procedure because information from the validation folds can influence the preprocessing. A Pipeline containing scaling and PCA inside each fold would give a stricter

estimate of generalization. For a lab experiment, the current code is still useful for understanding the comparison, but this point should be kept in mind when interpreting the numbers.

Another point is that the confusion matrices and ROC curves are produced after fitting the selected estimators on the full dataset. They are therefore not held-out test results. The cross-validation F1 values are the more useful numbers for comparing the models in this notebook.

12 Observation Questions

1. How does Bagging reduce variance?

Bagging trains several versions of a base model on different bootstrap samples and combines their predictions. A single decision tree can change a lot when a few training samples change. Combining many trees makes the final prediction more stable, so the variance is reduced.

In this notebook, Random Forest is the main bagging-style model. It uses multiple decision trees instead of relying on just one tree.

2. How does Boosting address model bias?

Boosting builds models one after another. Later models pay more attention to mistakes made earlier. Because the new models try to correct old errors, the final classifier can become stronger than a weak learner used alone.

Here, AdaBoost, Gradient Boosting and XGBoost are used for this purpose. Their results also show that adding more estimators does not automatically guarantee a better score.

3. Why does stacking benefit from heterogeneous models?

Different models make different kinds of decisions. KNN looks at nearby samples, Naive Bayes uses probability assumptions, and the Decision Tree uses feature-based splits. The Logistic Regression meta learner can combine these outputs instead of depending on one model type.

That is the main reason stacking can work well even when each base model is not the single best model by itself.

4. Which approach performed best in this run and why?

For the 5-fold F1 comparison, Logistic Regression is the best model on the original features with an average F1 of 0.9835. With PCA, it remains the best at 0.9834. SVM is almost the same. So the strongest result in this notebook comes from Logistic Regression rather than the ensemble models.

The useful result is not just the winning number. The comparison shows that PCA can reduce the input from 30 features to 10 while keeping the best F1 almost unchanged. That can make the data representation simpler without a large loss in the models that performed well here.

13 Conclusion

In this experiment, ten classifiers were compared on the Wisconsin Diagnostic Breast Cancer dataset using stratified 5-fold cross-validation. The main score used in the notebook was F1-score. PCA was also tested to check whether 30 features could be reduced without hurting the model performance.

The PCA analysis selected 10 principal components and retained 95.16% of the variance. Logistic Regression gave the highest average F1 without PCA (0.9835) and stayed almost unchanged after PCA (0.9834). Gradient Boosting showed the largest positive change after PCA, while Naive Bayes showed the largest drop.

Overall, the experiment gave a practical view of model comparison, ensemble learning and dimensionality reduction. The important takeaway is that PCA is useful for reducing dimensions, but its effect depends on the classifier. Also, validation must be done carefullly so that preprocessing does not leak information across folds.

14 References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] Scikit-learn, "Ensemble methods documentation," Online. Available: https://scikit-learn.org/stable/modules/ensemble.html

[3] Scikit-learn, "PCA documentation," Online. Available: https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

[4] Scikit-learn, "Breast cancer dataset," Online. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html

[5] UCI Machine Learning Repository, "Breast Cancer Wisconsin (Diagnostic) Dataset," Online. Available: https://archive.ics.uci.edu/